Polyformer: Embracing the Bitter Lesson

by Mostafa 'Neo' Mohsenvand, PhD

Complex Learning Group

Apr 11 2023

Scaled Dot Product Self-Attention (SDSA)

The transformer models have revolutionized machine learning, emerging as a key component of successful applications in natural language processing and computer vision. The central mathematical operation in transformer models is the scaled dot product self-attention (SDSA) algorithm, which in fact is a linear layer with quadratic complexity unlike the ordinary linear layer (LL) that enjoys linear complexity. SDSA has a computational complexity of $O(n^2 d_k h)$ in time and $O(n^2 h)$ in space. Here n is the length of the input sequence, h is the number of heads and d_k is the dimensionality of key, query and value vectors. The key difference of SDSA with the linear layer is the calculation of the attention scores which consists of the dot product of every query vector with every key vector in the sequence. The result of this calculation is a matrix. Here is the equation that is usually used to discuss the SDSA block.

$$\text{Attention}(K, Q, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

The term QK^T is an outer product between the sequence of queries Q and the sequence of keys K. The outer product is usually computed between two vectors, not two vector sequences. Here it is implied that the usual arithmatic multiplication is replaced by the inner product between vectors in the sequence. To make this explicit, Let's rewrite the equation in a way to describe the relationship between an input and output sequence. First we compute the components of the attention matrix by obtaining the pairwise inner products between all (query,key) pairs.

$$A_{ij} = Q_i \cdot K_j$$

Then we compute the contextualized value vectors y_i by recombining the value vectors using the normalized attention scores:

$$Y_i = \sum_j \underset{i}{\text{softmax}}(\frac{A_{ij}}{\sqrt{d_k}})V_j$$

Here we used $\underset{i}{\text{softmax}}$ to indicate that the softmax is computed over all components with a fixed $i, A_{i1}, A_{i2}, \dots, A_{in}$. This notation will come in handy later.

Motivation

Most research in transformers has focused on sub-quadratic approximations of SDSA to achieve faster and more scalable algorithms. In this work, we propose exploring the oposite direction by hypothesizing that the success of transformer models is due to the increased computation time and space facilitated by the $O(n^2)$ complexity of SDSA operation. Our hypothesis is supported by the lack of success of so called "Efficient Transformers" in practice which consistently under-perform the exact $O(n^2)$ models. Instead of modifying SDSA for lower complexity, we explore a generalization of SDSA to higher complexities by introducing a multilinear form self-attention operator that models higher-order interaction terms amongst the elements of the input sequence. The new operator computes attention scores for larger tuples of tokens,

effectively leading to $O(n^3)$, $O(n^4)$, and higher complexities. **We hypothesise that the resulting models will exhibit more expression and comprehension power per number of parameters, layers, attention heads and model dimensions compared to the SDSA algorithm.**

Our work is inspired by Rich Sutton's "The bitter lesson," which emphasizes that real progress in AI often happens due to the availability of more computational resources rather than developing more sophisticated models.

Generalized Dot Product

To generalize the SDSA algorithm to higher complexities, we need to generalize the dot product so that it can behave similar to the ordinary dot product but accept 3 or more operands instead of two. One can think of several candidates for the generalized dot product. To narrow down the search space, it is useful to consider the intuition behind the use of dot product in Attention mechanisms. When the dot product between two word embeddings is high, it means that there are dimensions or contexts where both vectors have a high value. Since each word can have several meanings, finding and amplifying common contexts amongsts the words can narrow down the meaning of the words. In other words, the job of the dot product is discovering and amplifying the right context in which the words are being used. It is insightful to break down dot product into two steps of 1) elementwise multiplication of the two input vectors and 2) summing over the components of the resulting vector.

$$V \cdot U = \sum_i V_i U_i$$

In this light, the generalization to more operands becomes trivial. We define the generalized dot product (GDP) of N vectors as:

$$\mathrm{GDP}(V^1, V^2, \dots V^k) = \sum_{i=1}^{d} \left(\prod_{j=1}^{k} V_i{}^j \right)$$

which reduces to dot product for $k = 2$. Another way to understand this generalization is the following representation of the dot product based on outer product:

$$V \cdot U = \mathrm{Tr}(V \otimes U)$$

Which leads to the natural generalization in the form of

$$\mathrm{GDP}(V^1, V^2, \dots V^k) = \mathrm{Tr}\left(\bigotimes_{j=1}^{k} V^j \right)$$

Here the trace operator $\mathrm{Tr}(\cdot)$ sums over the diagonal of the resulting tensor.

Yet another way of looking at this operation is through the lens of tensor contraction and multilinear forms. A multilinear form is a type of function that takes multiple vectors as input and produces a scalar as output. Specifically, a multilinear form is a function that is linear in each of its arguments, meaning that if any one of its arguments is scaled or replaced by a linear combination of vectors, the output of the function is scaled or replaced by the same linear combination.

For example, a bilinear form (e.g. dot product) is a multilinear form that takes two vectors as input and produces a scalar as output. A trilinear form takes three vectors as input, and so on. A general multilinear form can take any number of vectors as input.

Multilinear forms are important in many areas of mathematics and physics, including linear algebra, differential geometry, and quantum mechanics. Higher order multilinear forms naturally arrise in the study of many-body systems in statistical mechanics and Feynman diagrams in quantum field theory where once desires to compute higher order interaction terms.

Polyattention

Equiped with the GDP operator, we can generalize the attention matrix of SDSA as the outer product of a sequence of queries Q with multiple sequences of keys $K^1, K^2, \ldots, K^N$ where the implied multiplication is replaced by the GDP operator:

$$A_{ijk\cdots z} = \text{GDP}(Q_i, K^1_j, K^2_k, \ldots, K^N_z)$$

This can be written more explicitly as the dot prodoct of Q_i with the elementwise product of $K^1_j, K^2_k, \ldots, K^N_z$ as follows.

$$A_{ijk\cdots z} = Q_i \cdot K^1_j \odot K^2_k \odot \cdots \odot K^N_z$$

Note that here the final summation is implied by the ordinary dot prodoct after Q_i. This operation will have the time and space complexity of $O(n^{N+1})$ where n is the length of the sequence and N is the number of different kinds of key vectors. For $N = 1, A$ is identical to the attention matrix of SDSA.

The next piece of the puzzle is figuring out how to use this polyattention tensor to weigh the recombination of values. One might naively decide to sum over the last $N - 1$ axss of A to obtain a matrix for weighing value vectors. However, this approach misses the point and leads to redundent terms in the final sum. For example, let's consider the case for a model with cubic complexity where $A_{ijk} = Q_i \cdot K^1_j \odot K^2_k$ measures the influence of the tuple of j'th and k'th token on the i'th token and since the K^1 and K^2 sequences are obtained through two different linear layers, $A_{ijk} \neq A_{ikj}$. This property is precisely the source of improved representational power that is achieved by increasing the complexity of the model and summing over these components will remove this distinction and defeat the purpose. The A_{ijk} should give the weight of influence of **ordered tuple** of j'th and k'th tokens on the i'th token. But how can we preserve the order when producing the output vectors? From a tensor algebra perspective, the extra indices of A beg for a higher dimensional tensor rather than a sequence of vectors. Here we propose introducing new value sequences $V^1, V^2, \ldots, V^N$ and constructing an outer product of these sequences with elementwise multiplication. Let's call the resulting tensor D with components computed as follows.

$$D_{jk\cdots z} = V^1_j \odot V^2_k \odot \cdots \odot V^N_z$$

Here the value of $D_{jk\cdots z}$ for a specific choice of $j, k, \ldots, z$ is a vector obtained by the elementwise multiplication of $V^1_j, V^2_k, \ldots, V^N_z$ vectors. The fact that each of the $V^1, V^2, \ldots, V^N$ sequences have been obtained from a different linear layer, ensures the preservation of order and allow the model to take advantage of the added complexity.

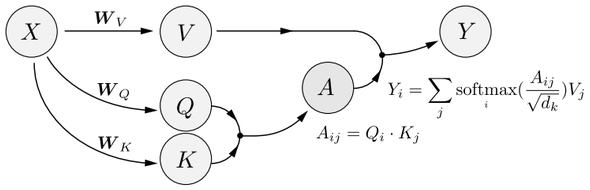
Now it is straightforward to compute the final output:

$$Y_i = \sum_{j,k,\cdots,z} \underset{i}{\text{softmax}}\left(\frac{A_{ijk\cdots z}}{\sqrt{d_k}}\right) D_{jk\cdots z}$$

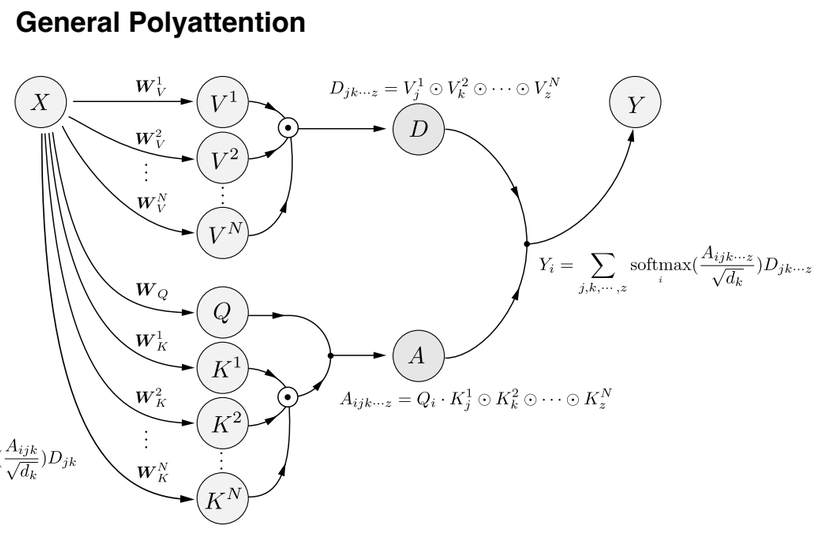
Note that $\underset{i}{\text{softmax}}$ is taken over all the elements of $A_{ijk\cdots z}$ for a fixed i.

The Figure below compares the computation graph of SDSA with the cubic and the general polyattention model and summarizes this section.

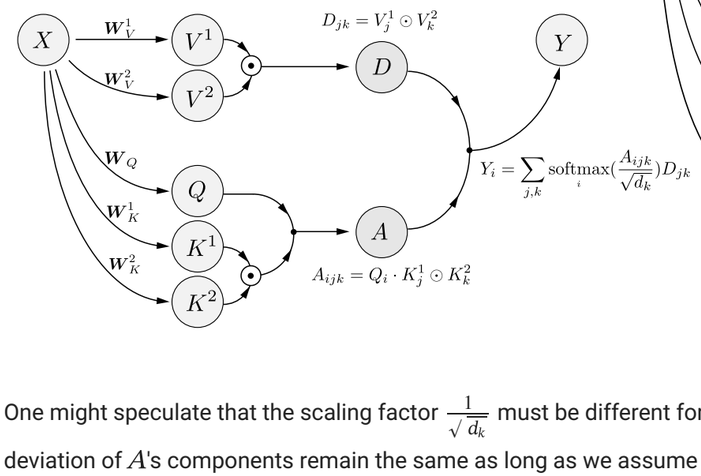
Scaled Dot-Product Self-Attention



General Polyattention



Cubic Polyattention



One might speculate that the scaling factor $\frac{1}{\sqrt{d_k}}$ must be different for the cases with higher complexity $N > 2$ but in reality, the standard deviation of A's components remain the same as long as we assume zero mean and unit variance for Q and $K^1, K^2, \ldots, K^N$ vectors. The proof can be obtained easily through induction.

Implementation

```
import torch
from torch import nn
from torch import functional as F
from torch import optim
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader
from tqdm import tqdm
import math
import matplotlib.pyplot as plt
%matplotlib inline
```

```
if torch.cuda.is_available():
    device = torch.device("cuda")
```

```
    print('GPU: ', torch.cuda.get_device_name(0))

else:
    device = torch.device("cpu")
    print('No GPU available')
```

Here *k* indicates the complexity of the model and it can be set any integer between 2 (ordinary self-attention) and 26.

```
def polyattention_scores(*tensors):
    k = len(tensors)
    if k < 2:
        raise ValueError("At least two tensors are required")

    # Build the einsum string for tensor contraction or generalized inner product (element-wise multiplication of all input vectors and then sum)
    # this block builds a string in the format of "bAc, bBc, ..., bKc -> bAB...K" which performs the tensor contraction


    input_str = ','.join(f'b{chr(ord("A") + i)}c' for i in range(k))
    output_str = 'b' + ''.join(f'{chr(ord("A") + i)}' for i in range(k))
    einsum_str = f'{input_str}->{output_str}'

    # Perform the einsum operation
    result = torch.einsum(einsum_str, *tensors)

    # Divide the result by the square root of the number of channels (the standard deviation is sqrt(d_k) regardless of the complexity)
    channel_size = tensors[0].shape[-1]
    result = result / (channel_size ** 0.5)

    # Apply the softmax function to the last k-1 axes
    result = torch.softmax(result.reshape(result.shape[0], result.shape[1], -1), dim=-1).reshape(result.shape)
    return result


def polyattention_values(*tensors):
    k = len(tensors)
    # the generalized value matrix is obtained by pairwise combination of all value vectors using elementwise multiplication
    # Build the einsum string for constructing a tensor of element-wise multiplication of all input vectors
    input_str = ','.join(f'b{chr(ord("A") + i)}c' for i in range(k))
    output_str = 'b' + ''.join(f'{chr(ord("A") + i)}' for i in range(k)) + 'c'
    einsum_str = f'{input_str}->{output_str}'

    # Perform the einsum operation
    result = torch.einsum(einsum_str, *tensors)

    return result

```

```
def polyattention(T_tensors, V_tensors, dropout):
    k = len(T_tensors)

    if len(V_tensors) != k - 1:
        raise ValueError("Mismatch in the number of T_tensors and V_tensors")

    # Calculate polyattention_scores and polyattention_values
    scores = polyattention_scores(*T_tensors)
    scores = dropout(scores)
    values = polyattention_values(*V_tensors)

    # Check if the sequence dimensions of scores and values tensors match the requirements
    if scores.shape[2:-1] != values.shape[1:-2]:
        raise ValueError("Mismatch in the sequence dimensions of scores and values tensors")

    # Build the einsum string
    scores_axes = ''.join(chr(ord('A') + i) for i in range(k))
    values_axes = ''.join(chr(ord('B') + i) for i in range(k - 1))
    input_str = f'b{scores_axes},b{values_axes}c'
    output_str = f'b{scores_axes[0]}c'
    einsum_str = f'{input_str}->{output_str}'

    # Perform the weighted sum of polyattention_values based on polyattention_scores
    result = torch.einsum(einsum_str, scores, values)

    return result, scores


class MultiheadPolyAttention(nn.Module):
    def __init__(self, embed_dim, heads=8, activation=None, dropout=0.1, k=2):
        super(MultiheadPolyAttention, self).__init__()
        self.heads = heads
        self.embed_dim = embed_dim
        self.k = k
        self.T_linear_layers = nn.ModuleList([nn.Linear(embed_dim, embed_dim) for _ in range(k)])
        self.V_linear_layers = nn.ModuleList([nn.Linear(embed_dim, embed_dim) for _ in range(k - 1)])

        if activation == 'relu':
            self.activation = nn.ReLU()
        else:
            self.activation = nn.Identity()
        self.dropout = nn.Dropout(dropout)

    def forward(self, inp):
        # inp: (batch_size, seq_len, embed_dim)
        batch_size, seq_len, embed_dim = inp.size()
        assert embed_dim == self.embed_dim
```

```
        T_tensors = [self._reshape_heads(self.activation(self.T_linear_layers[i](inp))) for i in range(self.k)]
        V_tensors = [self._reshape_heads(self.activation(self.V_linear_layers[i](inp))) for i in range(self.k-1)]
        out,scores = polyattention(T_tensors,V_tensors, self.dropout)
        out = self._reshape_heads_back(out)

        return out, scores

    def _reshape_heads(self, inp):
        # inp: (batch_size, seq_len, embed_dim)
        batch_size, seq_len, embed_dim = inp.size()

        reduced_dim = self.embed_dim // self.heads
        assert reduced_dim * self.heads == self.embed_dim
        out = inp.reshape(batch_size, seq_len, self.heads, reduced_dim)
        out = out.permute(0, 2, 1, 3)
        out = out.reshape(-1, seq_len, reduced_dim)

        # out: (batch_size * heads, seq_len, reduced_dim)
        return out

    def _reshape_heads_back(self, inp):
        # inp: (batch_size * heads, seq_len, reduced_dim) | reduced_dim = embed_dim // heads
        batch_size_mul_heads, seq_len, reduced_dim = inp.size()
        batch_size = batch_size_mul_heads // self.heads

        out = inp.reshape(batch_size, self.heads, seq_len, reduced_dim)
        out = out.permute(0, 2, 1, 3)
        out = out.reshape(batch_size, seq_len, self.embed_dim)

        # out: (batch_size, seq_len, embed_dim)
```

```
# Check if Dropout should be used after second Linear Layer
class FeedForward(nn.Module):
    def __init__(self, embed_dim, forward_expansion=1, dropout=0.1):
        super(FeedForward, self).__init__()
        self.embed_dim = embed_dim
        self.fc1 = nn.Linear(embed_dim, embed_dim * forward_expansion)
        self.activation = nn.GELU()
        self.fc2 = nn.Linear(embed_dim * forward_expansion, embed_dim)
        self.dropout = nn.Dropout(dropout)

    def forward(self, inp):
        # inp: (batch_size, seq_len, embed_dim)
        batch_size, seq_len, embed_dim = inp.size()
        assert embed_dim == self.embed_dim

```

```
        out = self.dropout(self.activation(self.fc1(inp)))
        # out = self.dropout(self.fc2(out))
        out = self.fc2(out)

        # out: (batch_size, seq_len, embed_dim)
        return out
```

```
class PolyformerBlock(nn.Module):
    def __init__(self, embed_dim, heads=8, activation=None, forward_expansion=1, dropout=0.1, k=2):
        super(PolyformerBlock, self).__init__()
        self.embed_dim = embed_dim
        self.norm1 = nn.LayerNorm(embed_dim)
        self.polyattention = MultiheadPolyAttention(embed_dim, heads, activation, dropout, k)
        self.norm2 = nn.LayerNorm(embed_dim)
        self.feed_forward = FeedForward(embed_dim, forward_expansion, dropout)

    def forward(self, inp):
        # inp: (batch_size, seq_len, embed_dim)
        batch_size, seq_len, embed_dim = inp.size()
        assert embed_dim == self.embed_dim

        res = inp
        out = self.norm1(inp)
        out, _ = self.polyattention(out)
        out = out + res

        res = out
        out = self.norm2(out)
        out = self.feed_forward(out)
        out = out + res

        # out: (batch_size, seq_len, embed_dim)
        return out
```

```
class Polyformer(nn.Module):
    def __init__(self, embed_dim, layers, heads=8, activation=None, forward_expansion=1, dropout=0.1, k=2):
        super(Polyformer, self).__init__()
        self.embed_dim = embed_dim
        self.poly_blocks = nn.ModuleList(
            [PolyformerBlock(embed_dim, heads, activation, forward_expansion, dropout, k) for i in range(layers)]
        )

    def forward(self, inp):
        # inp: (batch_size, seq_len, embed_dim)

        out = inp
```

```
        for block in self.poly_blocks:
            out = block(out)

        # out: (batch_size, seq_len, embed_dim)
        return out
```

0s completed at 9:40 PM